

MÔ PHỎNG CPU 8-CORE VỚI KIẾN TRÚC TẬP LỆNH RISC-V

Multiple Processing System sử dụng Verilog HDL

Thực hiện: Nhóm Học Viên

Báo Cáo Tiến Độ Dự Án Kiến Trúc Máy Tính

Ngày 17 tháng 6 năm 2026

Nội Dung Báo Cáo

Nội Dung Báo Cáo

Xu thế Đa Nhân (Multi-core Processing)

Giới hạn vật lý về tần số xung nhịp buộc kiến trúc máy tính hiện đại phải chuyển dịch sang kiến trúc đa nhân nhằm tối ưu hóa hiệu năng tính toán song song.

Tại sao chọn RISC-V?

- Kiến trúc tập lệnh (ISA) dạng mở, tinh gọn, có tính ứng dụng cao.
- Cấu trúc định dạng lệnh cố định, tạo điều kiện thuận lợi cho việc thiết kế phần cứng bằng ngôn ngữ mô tả phần cứng (Verilog HDL).

Nội Dung Báo Cáo

Tổng quan Tập lệnh RISC-V (RV32I)

RISC-V là gì?

Là một kiến trúc tập lệnh (ISA) mã nguồn mở dựa trên nguyên lý thiết kế máy tính tập lệnh rút gọn (RISC). Cho phép tự do thiết kế, tùy biến và tối ưu hóa phần cứng.

Giải nghĩa bộ lệnh cơ bản RV32I:

- **RV (RISC-V):** Tiêu chuẩn kiến trúc thế hệ thứ 5 từ UC Berkeley.
- **32 (32-bit):** Độ rộng thanh ghi và không gian địa chỉ 32-bit.
- **I (Integer):** Tập lệnh số nguyên cơ bản (gồm 40 lệnh). Đây là tập lệnh bắt buộc phải có cho mọi thiết kế chip RISC-V.

Mục tiêu lựa chọn

Đơn giản hóa thiết kế lõi đơn để tập trung tối ưu bộ điều khiển phân xử (Arbiter) và kết nối đa lõi (8-Core Interconnect).

Đặc trưng kỹ thuật RV32I

Các đặc trưng kỹ thuật:

- Không gian địa chỉ và độ dài lệnh cố định **32-bit**.
- Cơ chế **Load-Store**: Chỉ có lệnh Load/Store mới được truy cập Memory; các phép toán toán học chỉ thực thi trên Tập thanh ghi.
- **32 Thanh ghi đa năng** ($x0 \rightarrow x31$), trong đó $x0$ luôn cố định bằng số 0.
- 6 kiểu định dạng lệnh cơ bản: R, I, S, B, U, J.

Phân loại Tập thanh ghi (Register File)

- $x0$ (**zero**): Luôn bằng 0 (hằng số phần cứng).
- $x1$ (**ra**), $x2$ (**sp**): Địa chỉ trả về & Con trỏ Stack.
- $x3-x4$ (**gp-tp**): Con trỏ toàn cục và con trỏ tiểu trình.
- $x10-x17$ (**a0-a7**): Tham số truyền vào & Kết quả hàm.
- $x5-x7$, $x28-x31$ (**t0-t6**): Thanh ghi tạm thời.
- $x8-x9$, $x18-x27$ (**s0-s11**): Thanh ghi lưu giữ.

6 Kiểu định dạng lệnh cơ bản

	31	25	24	20	19	15	14	12	11	7	6	0		
R-type	funct7				rs2		rs1		funct3		rd		opcode	
I-type	imm[11:0]					rs1		funct3		rd		opcode		
S-type	imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode	
B-type	imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode	
U-type	imm[31:12]										rd		opcode	
J-type	imm[20 10:1 11 19:12]										rd		opcode	

Giải thích các trường

- **opcode**: Mã lệnh gốc (7 bit).
- **rd**: Thanh ghi đích.
- **funct3/7**: Mã mở rộng chức năng lệnh.
- **rs1/rs2**: Các thanh ghi nguồn.
- **imm**: Giá trị hằng số nạp trực tiếp.

Nội Dung Báo Cáo

Tập lệnh hiện thực trong dự án

Tập Lệnh Số Nguyên RV32I Subset

40 lệnh cơ bản được hỗ trợ:

- **Tính toán (ALU):** ADD/SUB, SLL/SRL, AND/OR/XOR, SLT (bao gồm cả dạng hằng số Imm như ADDI, ANDI,...).
- **Bộ nhớ:** LW (Load Word), SW (Store Word).
- **Điều khiển luồng:** BEQ, BNE, BLT, BGE, JAL, JALR.
- **Khác:** LUI, AUIPC.

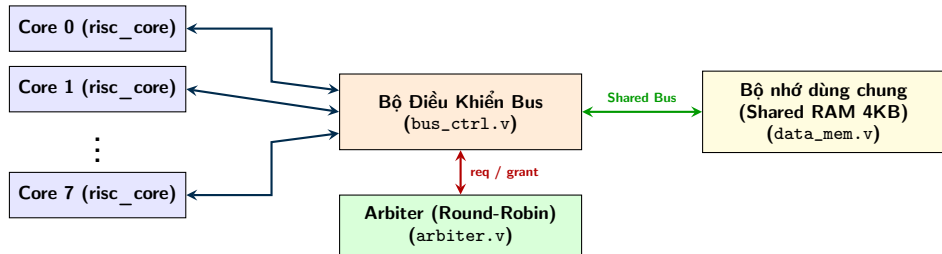
Đồng Bộ Đa Nhân (A-Extension)

Thiết kế riêng cho hệ thống 8-Core:

- **Độc quyền truy cập:** LR.W (Load-Reserved) và SC.W (Store-Conditional) để xây dựng cấu trúc đồng bộ (Mutex/Semaphore).
- **Phép toán nguyên tử (AMO):** AMOSWAP, AMOADD, AMOAND, AMOOR, AMOXOR, AMOMIN/MAX trực tiếp trên bộ nhớ dùng chung.

Nội Dung Báo Cáo

Cấu Trúc Tổng Thể Hệ Thống 8-Core



Hệ thống kết nối đa nhân: **8 lõi CPU** hoạt động song song độc lập, thực hiện phân xử quyền truy cập bộ nhớ thông qua bộ phân xử **Arbiter** và bộ định tuyến bus dữ liệu **Bus Controller**.

Nội Dung Báo Cáo

Cơ chế hoạt động của Pipeline Registers

Các thanh ghi đệm hoạt động như các "vách ngăn" đồng bộ bằng xung nhịp Clock:

Đồng bộ hóa & Dịch chuyển

- **Lưu giữ trạng thái:** Đảm bảo dữ liệu của lệnh trước không bị đè bởi lệnh sau khi đang xử lý song song.
- **Xếp hàng tín hiệu:** Tín hiệu điều khiển từ bộ Decode (`control_unit`) được đẩy dọc theo các thanh ghi đệm (`id_ex_reg_write`, `ex_mem_mem_read`,...) để thực thi đúng thời điểm.

Đường đi tắt (Forwarding Paths)

Bao vây các thanh ghi đệm để giải quyết xung đột dữ liệu:

- Bơm kết quả trực tiếp từ ngõ ra ALU (`ex_mem_result`) hoặc dữ liệu ghi ngược (`mem_wb_write_data`).
- Đưa thẳng về ngõ vào ALU tầng **EX** thông qua các bộ MUX lớn, giảm thiểu số chu kỳ bị Stall.

Bộ Giải Mã Lệnh (control_unit.v)

Bộ điều khiển control_unit.v là bộ giải mã trung tâm (decoder) hoạt động tại tầng **Decode (ID)**:

Giải mã tín hiệu điều khiển

Từ mã lệnh 32-bit, CU trích xuất opcode, funct3, funct7 để sinh:

- **Đường truyền dữ liệu:** reg_write, alu_src, mem_to_reg.
- **Điều khiển ALU:** alu_control[3:0] (ADD, SUB, SLL, SRL, AND, OR, XOR, SLT).
- **Rẽ nhánh & Nhảy:** branch, branch_type, jump, jalr.

Tín hiệu đa nhân đặc biệt

Hỗ trợ các tín hiệu điều khiển truy cập bộ nhớ và đồng bộ:

- mem_lr / mem_sc: Lệnh nguyên tử Load-Reserved / Store-Conditional.
- mem_amo: Kích hoạt phép toán nguyên tử trực tiếp trên bộ nhớ.
- amo_op[3:0]: Chọn phép toán AMO (swap, add, logic, min/max).

Bộ tạo hằng số (Immediate Generator): Giải mã và căn lề dấu (sign-extend) hằng số imm cho các định dạng lệnh I, S, B, U, J.

Quản lý xung đột trong Pipeline (hazard_unit)

Quá trình chạy gộp đầu lệnh (Pipelining) sinh ra các xung đột dữ liệu và điều khiển, được xử lý trực tiếp bên trong `risc_core.v`:

1. Xung đột dữ liệu (Data Hazard)

Xảy ra khi lệnh sau cần đọc giá trị thanh ghi mà lệnh trước chưa ghi xong.

- **Mạch Forwarding (Bơm tắt):** Chuyển kết quả trực tiếp từ ngõ ra EX/MEM hoặc MEM/WB về ngõ vào ALU mà không cần chờ Write Back.
- **Load-use Stall:** Với lệnh Load, kích hoạt tín hiệu `load_use_stall` để hoãn Pipeline 1 chu kỳ khi lệnh kế sau sử dụng kết quả Load.

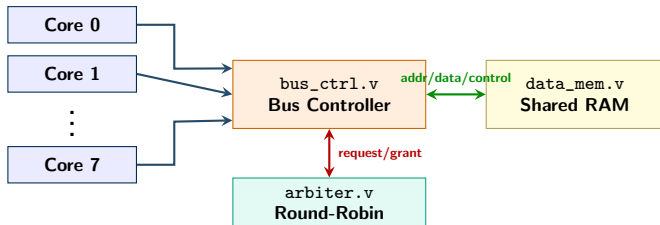
2. Xung đột điều khiển (Control)

Xảy ra khi các lệnh nhảy hoặc rẽ nhánh thay đổi dòng chảy địa chỉ PC.

- Khi tín hiệu `redirect_taken` kích hoạt (do Branch hoặc Jump).
- **Cơ chế Flush (Xóa):** Reset các thanh ghi đệm tầng nạp sai lệnh (`if_id_valid <= 0`, `id_ex_valid <= 0`), đưa các tín hiệu điều khiển về 0 (tạo lệnh rỗng NOP).

Vai Trò Của Interconnect Trong Hệ Thống 8 Core

Khi 8 core cùng cần đọc/ghi data_mem, interconnect đóng vai trò cầu nối để chọn đúng core, đúng địa chỉ và trả phản hồi về đúng nơi.



Nhóm tín hiệu chính

- Core yêu cầu: `core_mem_req`, `core_addr`, `core_wdata`.
- Kiểu giao dịch: `core_we`, `core_re`, `core_lr`, `core_sc`, `core_amo`.
- Phản hồi: `core_ready`, `core_rdata`, `core_sc_result`.

Ý chính

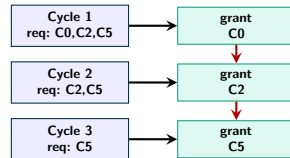
Interconnect không xử lý hazard pipeline; nó cấp quyền truy cập RAM và tạo tín hiệu sẵn sàng để core tiếp tục chạy.

Arbiter Round-Robin: Chia Quyền Truy Cập Bus

arbiter.v nhận vector request[7:0] và chọn một core được cấp bus trong mỗi lượt cấp quyền.

Cơ chế cấp quyền

- grant[7:0] là one-hot, chỉ core thắng arbitration được bật bit.
- grant_valid báo grant hợp lệ trong một chu kỳ.
- grant_id cho bus_ctrl biết core nào đang được phục vụ.
- priority_ptr dịch sang core kế tiếp sau mỗi grant.
- last_granted tránh cấp lại ngay cho cùng request cũ.



Mục tiêu

Chia sẻ RAM công bằng hơn giữa các core, giảm nguy cơ một core chiếm bus liên tục.

Read

- Bật `dm_re` và đưa `dm_addr` sang RAM.
- Nhận `dm_rdata` từ memory interface.
- Ghi dữ liệu vào `core_rdata[grant_id]` và bật `core_ready`.

Write

- Bật `dm_we`, đưa `dm_addr` và `dm_wdata`.
- `data_mem` ghi dữ liệu theo cạnh clock.
- `bus_ctrl` báo `core_ready` để core kết thúc giao dịch.

Trong thiết kế hiện tại, read ở memory interface là combinational/async; write là synchronous trong `data_mem`.

Sau khi arbiter cấp quyền, `bus_ctrl.v` chọn tín hiệu của core thắng và điều khiển giao tiếp với `data_mem`.

Atomic LR/SC Và AMO Trong Interconnect

Interconnect có primitive phần cứng cho đồng bộ đa lõi thông qua reservation và phép toán nguyên tử trên RAM dùng chung.

LR/SC Reservation

- LR.W: đọc memory và đặt `reservation_valid[core]` tại `reservation_addr[core]`.
- SC.W thành công nếu reservation còn đúng địa chỉ; khi đó RAM được ghi và kết quả trả về là thành công.
- Nếu core khác ghi cùng địa chỉ, reservation bị invalidate; SC.W sau đó thất bại.

AMO

- AMO đọc giá trị cũ từ RAM.
- `bus_ctrl` tính giá trị mới theo `amo_op`: add, swap, and, or, xor, min/max.
- RAM nhận giá trị mới; core nhận lại old value qua `core_rdata`.

Đây là nền tảng phần cứng để xây dựng cơ chế đồng bộ; slide này không trình bày như một mutex/semaphore phần mềm hoàn chỉnh.

Testbench Và Kết Quả Kiểm Chứng Interconnect

Kiểm chứng tập trung ở tb/interconnect_tb.v, tách riêng khỏi pipeline core để kiểm tra trực tiếp arbiter, bus_ctrl và data_mem.

Test	Mục tiêu kiểm chứng	Kết quả
Round-Robin Write/Read	4 core ghi/đọc các địa chỉ khác nhau qua bus dùng chung.	PASS
LR/SC success	Core thực hiện LR.W rồi SC.W khi không có core khác ghi chen vào.	PASS
LR/SC fail	Core khác ghi cùng địa chỉ làm reservation bị xóa trước SC.W.	PASS
AMOADD	Hai core cộng nguyên tử, RAM cuối cùng nhận đúng tổng giá trị.	PASS

Kết quả mô phỏng

SUMMARY: 10 PASSED, 0 FAILED

Tín hiệu nên mở trong GTKWave

core_mem_req, arb_request, arb_grant, grant_id, dm_addr, dm_we, dm_re, core_ready, core_sc_result.